

Министерство образования и науки РФ

Федеральное государственное автономное образовательное учебное
учреждение высшего образования “Санкт-Петербургский
национальный исследовательский университет ИТМО”

Факультет прикладной информатики

Лабораторная работа №4

По дисциплине: Проектирование и реализация баз данных

Выполнил:

Ситдилов Рафаэль Ильдусович

Группа: К3240

Проверила:

Говорова Марина Михайловна

Г. Санкт-Петербург, 2026г.

Задание:

Вариант 14. БД «Служба заказа такси»

Описание предметной области: Система должна фиксировать все вызовы такси и распределять их между водителями.

Каждому водителю ежедневно начисляется заработная плата в зависимости от количества вызовов и их тарифа (50% от заработанной им суммы). Автомобили могут быть собственностью компании или таксиста.

Заказ принимает дежурный администратор и передает его водителю. В заказе фиксируется тип оплаты – наличными или онлайн. Если заказ оплачивается онлайн, то в системе хранится привязка к карте, с которой была совершена оплата.

В системе необходимо хранить график работы водителей.

Ежедневно действуют базовые тарифы на тип предоставляемых авто, но в зависимости от времени суток и ситуации на дорогах, цена может корректироваться.

БД должна содержать следующий минимальный набор сведений: Код сотрудника. ФИО сотрудника. Адрес сотрудника. № телефона сотрудника. Паспортные данные сотрудника. Должность сотрудника. Категория сотрудника. Наименование модели и марки автомобиля. Технические характеристики. Стран-производитель. Стоимость. Код тарифа. Наименование тарифа. Цена за километр. Код автомобиля. Госномер автомобиля. Год выпуска. Пробег. Дата последнего ТО. Дата вызова. Время посадки пассажира. Время высадки пассажира. Номер телефона пассажира. Откуда. Куда. Расстояние. Штраф за время ожидания (в минутах). Оплата (онлайн (при заказе) или наличными). Рекламация клиента на вызов.

Отчёт по Лабораторным работам 2, 3 размещён по ссылке:

 ЛР3 В14 467478 Ситдилов Р. И. К3240

Реализация

Для работы используется резервная копия Базы Данных из лабораторной работы 3.

В базе данных используются следующие таблицы:

- position — должности сотрудников;
- contract — договоры сотрудников;
- employee — сотрудники службы такси;
- client — клиенты;
- bank_card — банковские карты клиентов;
- car_model — модели автомобилей;
- car — автомобили;
- tariff_price — базовые стоимости тарифов;
- tariff — тарифы;
- orders — заказы такси.

1. Запросы

Запрос 1. Водитель, выполнивший максимальное количество заказов.

Вывести данные о водителе, который выполнил наибольшее количество заказов:

```
SELECT
    e.id_employee,
    e.full_name,
    e.phone,
    e.email,
    COUNT(o.id_order) AS order_count
FROM taxi.employee e
JOIN taxi.orders o ON e.id_employee = o.employee_id
GROUP BY
    e.id_employee,
    e.full_name,
    e.phone,
    e.email

HAVING COUNT(o.id_order) = (
    SELECT COUNT(o2.id_order)
    FROM taxi.orders o2
    GROUP BY o2.employee_id
    ORDER BY COUNT(o2.id_order) DESC
    LIMIT 1;
)
```

```
ORDER BY order_count DESC;
```

В результате выполнения запроса был найден сотрудник, выполнивший максимальное количество заказов.

Запрос 2. Автомобили старше среднего возраста

Вывести данные об автомобилях, возраст которых больше среднего возраста автомобилей в таксопарке.

```
SELECT
  c.id_car,
  c.model_name,
  c.ownership_status,
  c.car_age,
  c.production_year,
  c.engine_number
FROM taxi.car c
WHERE c.car_age > (
  SELECT AVG(car_age)
  FROM taxi.car
);
```

В результате выполнения запроса были получены автомобили, возраст которых превышает среднее значение по таблице car.

	id_car [PK] bigint	model_name character varying (20)	ownership_status character varying (10)	car_age numeric (4,1)	production_year integer	engine_number character varying (50)
1	1	Kia Rio	Компания	5.0	2019	ENG001
2	5	Skoda Octavia	Водитель	6.0	2018	ENG005

Запрос 3. Количество заказов каждого клиента

Определить, сколько раз каждый клиент воспользовался услугами таксопарка.

```
SELECT
  c.id_client,
  c.client_name,
  c.account,
  COUNT(o.id_order) AS order_count
FROM taxi.client c
LEFT JOIN taxi.orders o ON c.id_client = o.client_id
```

```
GROUP BY
  c.id_client,
  c.client_name,
  c.account
ORDER BY order_count DESC;
```

Запрос позволяет получить список клиентов и количество заказов.

	id_client [PK] bigint 	client_name character varying (80) 	account character varying 	order_count bigint 
1	1	Иван Петров	ivan_petrov	1
2	3	Олег Иванов	oleg_ivanov	1
3	5	Дмитрий Кузнецов	dmitry_kuznetsov	1
4	4	Мария Соколова	maria_sokolova	1
5	2	Анна Смирнова	anna_smirnova	1

Запрос 4. Клиент с максимальным количеством заказов

Вывести данные клиента, который воспользовался услугами таксопарка максимальное число раз.

```
SELECT
  c.id_client,
  c.client_name,
  c.account,
  c.rating,
  COUNT(o.id_order) AS order_count
FROM taxi.client c
JOIN taxi.orders o ON c.id_client = o.client_id
GROUP BY
  c.id_client,
  c.client_name,
  c.account,
  c.rating
HAVING COUNT(o.id_order) = (
  SELECT COUNT(o2.id_order)
  FROM taxi.orders o2
  GROUP BY o2.client_id
  ORDER BY COUNT(o2.id_order) DESC
```

```

LIMIT 1
)
ORDER BY order_count DESC;

```

В результате выполнения запроса был найден клиент с наибольшим количеством заказов.

	id_client [PK] bigint	client_name character varying (80)	account character varying	order_count bigint	driver_count bigint
1	1	Иван Петров	ivan_petrov	1	1
2	2	Анна Смирнова	anna_smirnova	1	1
3	3	Олег Иванов	oleg_ivanov	1	1
4	4	Мария Соколова	maria_sokolova	1	1
5	5	Дмитрий Кузнецов	dmitry_kuznetsov	1	1

Запрос 5. Водитель, выполнивший заказ с максимальной стоимостью

Вывести данные водителя, который выполнил заказ с максимальной стоимостью поездки.

```

SELECT
    e.id_employee,
    e.full_name,
    e.phone,
    e.email,
    o.id_order,
    o.trip_cost
FROM taxi.employee e
JOIN taxi.orders o ON e.id_employee = o.employee_id
WHERE o.trip_cost = (
    SELECT MAX(trip_cost)
    FROM taxi.orders
);

```

Запрос использует подзапрос для определения максимальной стоимости поездки.

	id_employee bigint	full_name character varying (120)	phone character varying (20)	email character varying (100)	id_order bigint	trip_cost integer
1	1	Алексей Иванов	+79000000001	ivanov.driver@mail.ru	4	1800

Запрос 6. Клиенты, которые всегда ездили с одним водителем

Необходимо вывести клиентов, которые пользовались услугами только одного и того же водителя.

```
SELECT
  c.id_client,
  c.client_name,
  c.account,
  COUNT(o.id_order) AS order_count,
  COUNT(DISTINCT o.employee_id) AS driver_count
FROM taxi.client c
JOIN taxi.orders o ON c.id_client = o.client_id
GROUP BY
  c.id_client,
  c.client_name,
  c.account
HAVING COUNT(DISTINCT o.employee_id) = 1;
```

Запрос группирует заказы по клиентам и отбирает только тех клиентов, у которых количество различных водителей равно одному.

	id_client [PK] bigint	client_name character varying (80)	account character varying	order_count bigint	driver_count bigint
1	1	Иван Петров	ivan_petrov	1	1
2	2	Анна Смирнова	anna_smirnova	1	1
3	3	Олег Иванов	oleg_ivanov	1	1
4	4	Мария Соколова	maria_sokolova	1	1
5	5	Дмитрий Кузнецов	dmitry_kuznetsov	1	1

Запрос 7. Автомобили в заказах со стоимостью выше средней

Вывести автомобили, которые использовались в заказах со стоимостью выше средней стоимости поездки.

```
SELECT DISTINCT
  car.id_car,
  car.model_name,
  car.ownership_status,
  car.car_age,
  car.production_year,
  car.engine_number,
  o.trip_cost
FROM taxi.car car
```

```
JOIN taxi.orders o ON car.id_car = o.car_id
WHERE o.trip_cost > (
    SELECT AVG(trip_cost)
    FROM taxi.orders
)
ORDER BY o.trip_cost DESC;
```

В результате выполнения запроса были получены автомобили, использованные в заказах, стоимость которых превышает среднюю стоимость поездки.

	id_car bigint	model_name character varying (20)	ownership_status character varying (10)	car_age numeric (4,1)	production_year integer	engine_number character varying (50)	trip_cost integer
1	4	BMW 5 Series	Компания	2.0	2022	ENG004	1800
2	3	Toyota Camry	Компания	3.0	2021	ENG003	1100

2. Представления

Представление 1. Сведения о незанятых водителях

Создать представление, содержащее сведения о водителях, которые не заняты выполнением заказа в заданный момент времени.

```
CREATE VIEW taxi.free_drivers AS
SELECT
    e.id_employee,
    e.full_name,
    e.phone,
    e.email,
    e.rating
FROM taxi.employee e
WHERE e.id_employee NOT IN (
    SELECT o.employee_id
    FROM taxi.orders o
    WHERE TIMESTAMP '2026-06-15 10:00'
        BETWEEN o.boarding_start AND o.actual_arrival
);
```

Просмотр содержимого:

```
SELECT * FROM taxi.free_drivers;
```

В результате выполнения запроса были получены сведения о сотрудниках, которые не выполняют заказ в указанный момент времени.

	id_employee bigint	full_name character varying (120)	phone character varying (20)	email character varying (100)	rating numeric (2,1)
1	1	Алексей Иванов	+79000000001	ivanov.driver@mail.ru	4.8
2	2	Павел Смирнов	+79000000002	smirnov.driver@mail.ru	4.6
3	3	Илья Кузнецов	+79000000003	kuznetsov.driver@mail....	4.9
4	4	Ольга Орлова	+79000000004	orlova.admin@mail.ru	4.5
5	5	Марина Волкова	+79000000005	volkova.dispatch@mail...	4.4

Представление 2. Зарплата водителей за выбранный день

Создать представление, содержащее сведения о зарплате водителей за выбранный день.

```
CREATE VIEW taxi.driver_salary_day AS
SELECT
  e.id_employee,
  e.full_name,
  COUNT(o.id_order) AS order_count,
  COALESCE(SUM(o.trip_cost), 0) AS total_income,
  COALESCE(SUM(o.trip_cost), 0) * 0.5 AS driver_salary
FROM taxi.employee e
LEFT JOIN taxi.orders o
  ON e.id_employee = o.employee_id
  AND DATE(o.order_datetime) = DATE '2026-06-15'
GROUP BY
  e.id_employee,
  e.full_name
ORDER BY driver_salary DESC;
```

Просмотр содержимого:

```
SELECT * FROM taxi.driver_salary_day;
```

В результате выполнения запроса были получены данные о количестве заказов, общей сумме поездок и рассчитанной зарплате водителей за выбранный день.

	id_employee bigint	full_name character varying (120)	order_count bigint	total_income bigint	driver_salary numeric
1	3	Илья Кузнецов	1	1100	550.0
2	1	Алексей Иванов	1	750	375.0
3	2	Павел Смирнов	1	620	310.0
4	5	Марина Волкова	0	0	0.0
5	4	Ольга Орлова	0	0	0.0

3. Запросы на модификацию данных

Были выполнены три запроса на модификацию данных с использованием подзапросов: INSERT, UPDATE и DELETE.

Запрос 1. INSERT с использованием подзапросов

Необходимо добавить нового сотрудника в таблицу employee.

```
INSERT INTO taxi.employee
(
    full_name,
    phone,
    email,
    contract_id,
    experience,
    rating
)
VALUES
(
    'Андрей Фёдоров',
    '+790000000006',
    'andrey_fedorov@gmail.com',
    (
        SELECT id_contract
        FROM taxi.contract
        ORDER BY start_date DESC, id_contract ASC
        LIMIT 1
    ),
    2.5,
    4.6
);
```

После выполнения запроса в таблицу был добавлен новый тестовый сотрудник с id_employee = 6.

Проверка результата:

```
SELECT * FROM taxi.orders
ORDER BY id_order;
```

Запрос 2. UPDATE с использованием подзапроса

Повысить рейтинг клиента, который имеет наибольшее количество заказов среди клиентов с рейтингом ниже 5.0.

```
UPDATE taxi.client
SET rating = LEAST(rating + 0.1, 5.0)
WHERE id_client = (
    SELECT o.client_id
    FROM taxi.orders o
    JOIN taxi.client c ON c.id_client = o.client_id
    WHERE c.rating < 5.0
    GROUP BY o.client_id
    ORDER BY COUNT(*) DESC
    LIMIT 1
);
```

Проверка результата:

```
SELECT * FROM taxi.client
ORDER BY id_client;
```

	id_client [PK] bigint	client_name character varying (80)	account character varying	rating numeric (2,1)
1	1	Иван Петров	ivan_petrov	4.9
2	2	Анна Смирнова	anna_smirnova	4.5
3	3	Олег Иванов	oleg_ivanov	4.2
4	4	Мария Соколова	maria_sokolova	5.0
5	5	Дмитрий Кузнецов	dmitry_kuznetsov	3.9

После выполнения запроса рейтинг клиента Иван Петров был изменён с 4.8 до 4.9.

Запрос 3. DELETE с использованием подзапроса

Удалить тестовый заказ с максимальным идентификатором.

```
DELETE FROM taxi.employee
WHERE id_employee = (
    SELECT id_employee
    FROM taxi.employee
    WHERE email = 'andrey_fedorov@mail.ru'
    LIMIT 1
);
```

Проверка результата:

```
SELECT * FROM taxi.employee
```

```
ORDER BY id_employee;
```

После выполнения запроса тестовый заказ с `id_employee = 6` был удалён, количество записей в таблице `orders` уменьшилось с 6 до 5.

4. Создание индексов и анализ планов выполнения запросов

Для анализа эффективности индексов использовалась команда `EXPLAIN ANALYZE`. Были рассмотрены простой и составной индексы.

Создать простой индекс для ускорения поиска заказов по клиенту.

```
EXPLAIN ANALYZE
SELECT *
FROM taxi.orders
WHERE client_id = 1;
```

После этого был создан простой индекс:

```
CREATE INDEX idx_orders_client_id
ON taxi.orders(client_id);
```

Затем тот же запрос был выполнен повторно:

```
EXPLAIN ANALYZE
SELECT *
FROM taxi.orders
WHERE client_id = 1;
```

	QUERY PLAN
	text
1	Seq Scan on orders (cost=0.00..1.06 rows=1 width=256) (actual time=0.027..0.029 rows=1.00 loop...
2	Filter: (client_id = 1)
3	Rows Removed by Filter: 4
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=16 read=1
7	Planning Time: 2.661 ms
8	Execution Time: 0.056 ms

Составной индекс

Необходимо создать составной индекс для ускорения поиска заказов по сотруднику и тарифу.

Запрос без индекса:

```
EXPLAIN ANALYZE
```

```
SELECT *
FROM taxi.orders
WHERE employee_id = 1
AND tariff_id = 3;
```

	QUERY PLAN	
	text	
1	Seq Scan on orders (cost=0.00..1.07 rows=1 width=256) (actual time=0.075..0.076 rows=1.00 loop...	
2	Filter: ((employee_id = 1) AND (tariff_id = 3))	
3	Rows Removed by Filter: 4	
4	Buffers: shared hit=1	
5	Planning Time: 0.263 ms	
6	Execution Time: 0.118 ms	

После этого был создан составной индекс:

```
CREATE INDEX idx_orders_employee_tariff
ON taxi.orders(employee_id, tariff_id);
```

Затем тот же запрос был выполнен повторно:

```
EXPLAIN ANALYZE
SELECT *
FROM taxi.orders
WHERE employee_id = 1
AND tariff_id = 3;
```

	QUERY PLAN	
	text	
1	Seq Scan on orders (cost=0.00..1.07 rows=1 width=256) (actual time=0.027..0.028 rows=1.00 loop...	
2	Filter: ((employee_id = 1) AND (tariff_id = 3))	
3	Rows Removed by Filter: 4	
4	Buffers: shared hit=1	
5	Planning:	
6	Buffers: shared hit=19 read=1	
7	Planning Time: 3.601 ms	
8	Execution Time: 0.047 ms	

Вывод:

В ходе выполнения лабораторной работы были разработаны SQL-запросы на выборку данных к базе данных «Служба заказа такси». Были созданы запросы с группировкой, сортировкой, агрегатными функциями, соединениями таблиц и подзапросами.

Также были созданы представления для отображения свободных водителей и расчёта зарплаты водителей за выбранный день. Были выполнены запросы на модификацию данных INSERT, UPDATE и DELETE с использованием подзапросов.

Для анализа производительности были созданы простой и составной индексы. С помощью команды EXPLAIN ANALYZE были получены планы выполнения запросов до и после создания индексов. В результате было установлено, что на малом объёме данных PostgreSQL может использовать последовательное сканирование таблицы даже при наличии индексов.

В результате выполнения лабораторной работы были получены практические навыки создания запросов, представлений, модификации данных, работы с индексами и анализа планов выполнения запросов в PostgreSQL.